Borg: Vulkan from vkcube Down to the Gates

Andreas Wendleder
Independent Researcher
Landshut, Germany
andreas.wendleder@proton.me

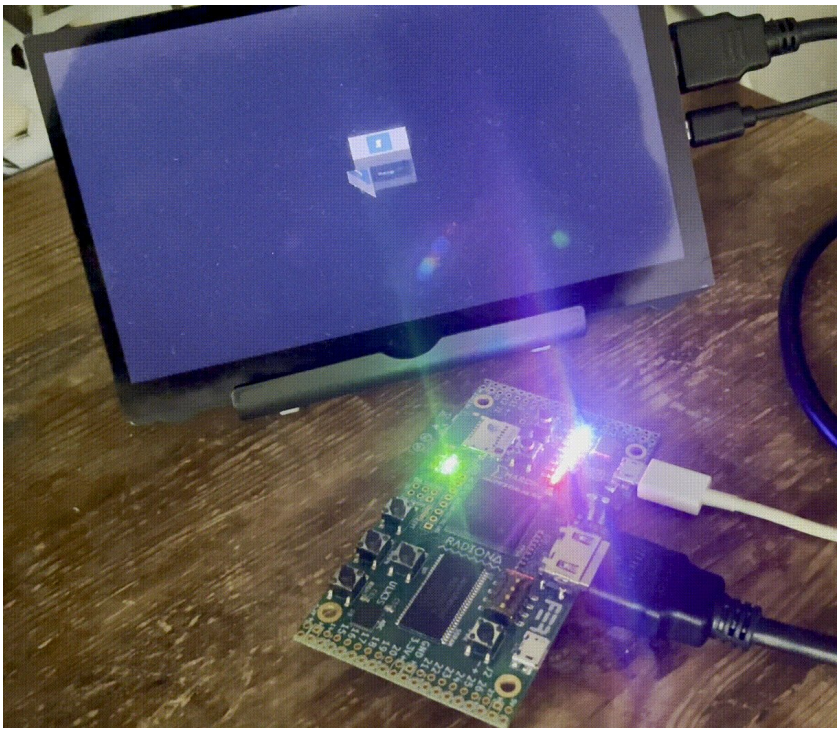


Figure 1: The unmodified Khronos `cube.c` rendered by Borg on a ULX3S board (Lattice ECP5-85K), shown over HDMI.

Abstract

Borg is an open-source GPU: Chisel RTL for a small FP32 shader core that runs 2x2 quads, with a tile-based rasterizer, a descriptor-based texture unit and MSAA, aimed at an ECP5 FPGA and a GF180 ASIC. Its Vulkan driver, borgvk, is a native Mesa ICD with its own NIR-based shader compiler. Unmodified Khronos vkcube renders on the FPGA, also self-hosted on Linux running on the board itself. This paper follows Vulkan 1.0 down to the gates: which API features forced hardware into the design, how a driver and compiler fit a GPU with 32 registers per lane, and how we run the Vulkan CTS directly against the RTL with a compiled simulator to find hardware bugs before tapeout.

Keywords

Vulkan, GPU architecture, open hardware, Mesa, conformance testing

1 Introduction

Borg is a GPU small enough for one person to read end to end: about 14,000 lines of Chisel for the GPU, a RISC-V CPU (Hutt) for the system around it, and a Vulkan driver in Mesa. The same Chisel source targets the ULX3S FPGA board and a GF180MCU ASIC slot on wafer.space. Everything is open source.[1]

The goal is not speed but completeness: a GPU that implements what Vulkan 1.0 actually requires, and a driver that runs real applications, starting with the unmodified Khronos `cube.c` (Figure 1). Figure 2 shows the stack. The driver, borgvk, is a native Mesa ICD modelled on v3dv: it uses Mesa's Vulkan runtime and compiles SPIR-V through NIR to Borg's ISA. Its output is a stream of command packets, which reaches one of three back ends: the FPGA over a serial link, a Linux DRM driver on the board itself, or a compiled simulator of the RTL that runs the Vulkan conformance test suite (CTS).

2 What Vulkan 1.0 asks of hardware

Several features of Borg exist only because the API demanded them. Each was cheaper to build than to work around. Table 1 summarizes them.

A graphics queue implies compute. Vulkan 1.0 requires every queue family with graphics to support compute. Borg therefore

[1] https://github.com/gonsolo/Borg, documentation at https://gonsolo.github.io/Borg/.

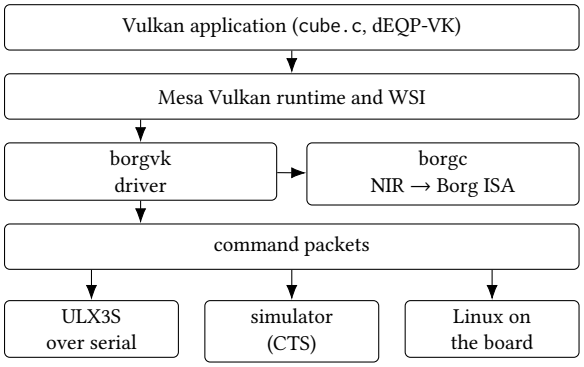


Figure 2: One driver, three back ends. All three consume the same packet stream; the simulator runs the RTL itself.

Table 1: Vulkan 1.0 requirements and the Borg hardware behind them.

Requirement	Borg mechanism
Compute on every graphics queue	workgroup dispatch, `BARRIER`, atomics
Derivatives, implicit LOD	2x2 quad on four lanes
Sampled images, samplers	descriptor-table texture unit, 51 formats
Colour attachment formats	raw attachments, blend by read-back
Depth, stencil, scissor, blend	fixed function in the tile pipeline
Occlusion queries	exact passing-sample counts
4x MSAA	multi-pass rendering per tile
Vertex input, topologies	hardware draw front end

dispatches workgroups on the same shader core, with a `BARRIER` instruction and atomics. This turned out to matter far beyond compute applications: every `glsl.builtin.precision` case in the CTS runs its shaders as compute dispatches, so the compute path gates thousands of shader tests.

Derivatives need quads. `dFdx` and `dFdy` are differences between neighbouring pixels, so the fragment core runs a 2x2 quad on four lanes, and implicit LOD in the texture unit uses the same quad. A one-lane core would be smaller and would return zero derivatives.

Descriptors become hardware tables. Instead of a fixed texture format, the texture unit reads descriptor tables from memory: 51 formats, mipmaps with implicit LOD, every address mode and border colour, trilinear filtering, depth compare, gather, `texelFetch` and offsets. The driver writes Vulkan's image and sampler state straight into these tables.

Formats and fixed function. Beyond a few native colour formats, every colour attachment is a raw attachment: the fragment shader packs the format and blends by reading the destination back. Depth, stencil, scissor, blending, exact occlusion query counts and 4x MSAA are in hardware. MSAA renders in several passes, which saved about 9% of the area compared with storing all samples at once.

The draw front end. A single register write renders a whole draw: primitive assembly for lists, strips, fans, indices, restart and instancing; one vertex shader run per triangle; varyings written and read back by two instructions; and a homogeneous 2D setup that needs no clipping, because vertices behind the eye are handled exactly.

```
fadd    rd rs1 rs2 f3   31..25=0x00 6..0=0x00
fmadd   rd rs1 rs2 rs3 f3   26..25=0 6..0=0x04
tex     rd rs1 rs2 rs3 f3   26..25=2 6..0=0x04
brnz    rs1 offset f3   31..25=0x28 6..0=0x00
expush  rs1 f3   31..25=0x2A 24..20=0 11..7=0 6..0=0x00
```

Figure 3: An excerpt of the opcode file. The decoder, the C header and the compiler's encoder are generated from it, so they cannot disagree.

3 A driver and compiler for 32 registers

borgvk follows the structure of other Mesa drivers: it builds on Mesa's Vulkan runtime for objects, synchronization and WSI, and submits through the runtime's queue. What is specific to Borg is the target.

The shader compiler, borgc, consumes NIR after Mesa's standard lowering. One instruction selector serves graphics and compute. The core has 32 registers per lane; constants live in a small uniform window that an instruction can read as one of its operands, so the compiler places constants in a pool and preloads what does not fit. Branches are PC-relative, so code does not depend on where it is loaded. Where a shader cannot be compiled correctly yet, borgc refuses it instead of guessing; a refused pipeline is an honest CTS failure, while a miscompiled one can pass by luck and fail later.

Two design decisions keep this manageable. The ISA lives in one file in the format of the RISC-V `riscv-opcodes` project (Listing 3), from which the hardware decoder, the C header and the compiler's encoder are generated. And every compiler change is checked against a corpus: recorded shaders are recompiled, and a change that should be a pure refactoring must produce identical binaries.

Three back ends. The packet stream goes to the ULX3S over a serial link, so a desktop Vulkan application renders on the FPGA; to the simulator described below; or, on the board itself, through a DRM render-node driver under Linux on the 64-bit Hutt CPU. In that last configuration `cube.c` runs on the board with no host computer at all. The driver code is the same in all three cases; only the transport changes.

4 Running the CTS against RTL

An FPGA or a gate-level simulation is far too slow to run a conformance suite with hundreds of thousands of cases. Borg's answer is a direct simulator: the GPU's RTL, without CPU or firmware, compiled to native code with CIRCT's arcilator, and driven by the same packet stream borgvk sends to the board. It runs at roughly 140,000 to 230,000 cycles per second, and many instances run in parallel. The CTS sees an ordinary Vulkan device.

Speed still decides what can be tested. The CTS shader-executor cases use one invocation per workgroup, which left three of the four quad lanes idle; packing four such workgroups into one quad cut an `atan2` vec4 case from over 40 minutes to about two.

Running the CTS against the RTL finds bugs in all layers. Examples: the texture unit returned one subtap instead of the average

Table 2: Selected CTS results on the direct simulator (CTS 1.4.6.2).

Scope	Cases	Pass	Fail
`dEQP-VK.api` (2026-10-05)	278,342	64,866	0
Graphics compiler corpus	2,293	728	12
Compute compiler corpus	3,990	3,711	258

at cube corners in `textureGather`, which surfaced against the CTS reference; integer copies through a floating-point move were flushed to zero when their bits looked like denormals; and branch targets in fragment programs landed one word early. Each was found and fixed before silicon.

Table 2 gives the current state. The `api` group runs without failures, the remaining cases being optional features reported as not supported. The compiler corpora (2026-10-20) are the CTS cases whose shaders reach borgc; the cases neither passing nor failing are not supported (graphics) or exceeded the 120 s limit per case (compute). The full Vulkan 1.0 must-pass list does not pass yet; it is being worked through group by group, with per-group results on the project's status page.

5 Lessons for a new implementation

- Bring up the `api` group first: it needs almost no shaders and settles the object model.
- Do compute early. It is mandatory, and the precision tests depend on it.
- Make the simulator fast before chasing failures; test throughput sets the pace of everything else.
- Refuse what the compiler cannot do. Silent miscompilation hides bugs that the CTS would otherwise report.
- Keep one source of truth for the ISA and register layouts; disagreements between copies were the cause of several bugs above.

The next steps are the full Vulkan 1.0 must-pass list, the remaining pipeline and GLSL groups, and a tapeout on wafer.space.

References

[1] J. Bachrach et al. Chisel: Constructing Hardware in a Scala Embedded Language. DAC 2012.
[2] CIRCT: Circuit IR Compilers and Tools. https://circt.llvm.org
[3] Khronos Group. Vulkan Conformance Test Suite (VK-GL-CTS). https://github.com/KhronosGroup/VK-GL-CTS
[4] The Mesa 3D Graphics Library. https://mesa3d.org
[5] RISC-V International. riscv-opcodes. https://github.com/riscv/riscv-opcodes
[6] Radiona. ULX3S FPGA board. https://radiona.org/ulx3s/